

# Lab-2 Worksheet Report

## CNN Training and Analysis on CIFAR-10

Dwivedi Jyoti Rajeshbhai (M25CSA010)

31 January 2026

### 1 Introduction

The objective of this laboratory experiment is to design, train, and analyze a Convolutional Neural Network (CNN) using the CIFAR-10 dataset. The focus of the assignment is not only on model performance, but also on understanding computational complexity, gradient behavior, and parameter update dynamics during training.

As part of this experiment, a custom data loader was implemented, FLOPs were computed for the selected CNN architecture, and detailed visualizations of gradient flow and weight updates were logged using Weights & Biases (WandB). The complete implementation and experiment tracking adhere strictly to the lab instructions.

### 2 Dataset Description

The CIFAR-10 dataset consists of 60,000 color images of size  $32 \times 32 \times 3$ , distributed evenly across 10 classes. The dataset is split into 50,000 training images and 10,000 test images.

To ensure stable training and better generalization, standard preprocessing techniques were applied, including normalization using dataset-specific mean and standard deviation (0.5, 0.5, 0.5). Data augmentation techniques were incorporated during training to improve robustness.

### 3 Custom DataLoader

A custom PyTorch Dataset and DataLoader pipeline was implemented instead of relying on a fully pre-built loader. This allowed finer control over preprocessing and batching.

The following transformations were applied:

- Random Rotation by 20 degrees, for data augmentation.
- Conversion of images to tensors
- Normalization with mean and standard deviation (0.5, 0.5, 0.5)

Batch shuffling was enabled for the training set to ensure better stochasticity and improved convergence behavior.

## 4 CNN Model Architecture

A custom CNN architecture was selected to balance representational capacity and computational efficiency. The model consists of:

- Three convolutional layers with increasing channel depth
  - Conv Layer 1: 3 input channels  $\rightarrow$  **32** output channels, kernel size  $3 \times 3$
  - Conv Layer 2: 32 input channels  $\rightarrow$  **64** output channels, kernel size  $3 \times 3$
  - Conv Layer 3: 64 input channels  $\rightarrow$  **128** output channels, kernel size  $3 \times 3$
- ReLU activation functions after each convolution for non-linearity
- MaxPooling layers with kernel size  $2 \times 2$  for spatial downsampling
- Fully connected layers with **10** output neurons for final classification

The final output layer produces logits for 10 classes corresponding to the CIFAR-10 categories. This architecture is well-suited for CIFAR-10 image resolution and allows effective feature extraction without excessive computational cost.

## 5 FLOPs and Parameter Analysis

The computational complexity of the model was estimated using a FLOPs profiling tool. A dummy input of size  $1 \times 3 \times 32 \times 32$  was used for estimation.

- Total number of trainable parameters: approximately **0.62** Million.
- Floating Point Operations (FLOPs): approximately **10.85** Million (Calculated via thop)

These values indicate that the model is computationally lightweight and suitable for small-scale image classification tasks such as CIFAR-10.

These metrics were logged to WandB to provide insight into the trade-off between model complexity and achieved performance. FLOPs analysis helps in understanding the feasibility of deploying the model in resource-constrained environments.

## 6 Training Configuration

The model was trained with the following configuration:

- Optimizer: Adam
- Loss Function: Cross-Entropy Loss
- Learning Rate: 0.001
- Batch Size: 64
- Number of Epochs: 30

The training process was performed on GPU. Training and validation metrics were logged at each epoch using WandB for systematic tracking and analysis.

## 7 Gradient Flow Visualization

To analyze training stability, gradient flow was visualized by tracking gradient norms for all trainable layers during backpropagation. These visualizations help identify issues such as vanishing or exploding gradients.

The logged plots showed that:

- Early layers maintained stable gradient magnitudes
- Deeper layers exhibited relatively smaller gradients

Overall, gradient flow remained well-behaved throughout training, indicating stable optimization dynamics.

## 8 Weight Update Flow Visualization

Weight update distributions were visualized using histograms logged to WandB. These plots illustrate how parameters evolve during training.

Observations from weight visualizations include:

- Gradual concentration of weights as training progressed
- No abnormal spikes or divergence in parameter values

This indicates controlled learning and absence of unstable parameter updates.

## 9 Results

The model demonstrated a consistent decrease in training loss and steady improvement in training accuracy across epochs. Validation accuracy improved initially and later stabilized, indicating reasonable generalization.

Key trends observed:

- Training loss decreased smoothly
- Training accuracy increased consistently (**84.29%**)
- Validation metrics showed moderate fluctuations
- Validation accuracy:(**88.67%**)
- Test accuracy:(**75.57%**)

Exact quantitative results and epoch-wise trends are available in the WandB dashboard.

## 10 Observations and Analysis

Based on the training curves, gradient flow, and weight update visualizations, the following observations were made:

- The model successfully learned discriminative features from CIFAR-10
- No severe gradient instability was observed
- Mild overfitting tendencies appeared in later epochs
- Computational complexity was reasonable for the achieved performance

These findings highlight the importance of monitoring internal training dynamics beyond accuracy alone.

## 11 WandB Reporting

All relevant metrics and visualizations were logged using Weights & Biases, including:

- Training and validation loss
- Accuracy curves
- FLOPs and parameter counts
- Gradient flow plots
- Weight update distributions

WandB enabled interactive inspection of training behavior and served as a central experiment tracking platform.

## 12 Submission Details

- **GitHub Repository:**

[https://github.com/Jyoti-Dwivedi-010/MLOps-Jyoti-Dwivedi-M25CSA010/tree/Jyoti-Dwivedi\\_M25CSA010\\_Lab2\\_worksheet](https://github.com/Jyoti-Dwivedi-010/MLOps-Jyoti-Dwivedi-M25CSA010/tree/Jyoti-Dwivedi_M25CSA010_Lab2_worksheet)

- **WandB Run:**

<https://wandb.ai/dwivedijyoti620-prom-iit-rajasthan/cifar10-lab2/runs/6g0tde6x>

## 13 Conclusion

This experiment provided practical insights into CNN training, computational analysis, and internal optimization dynamics. By combining performance metrics with gradient and weight flow visualizations, a deeper understanding of model behavior was achieved. The use of WandB significantly enhanced experiment transparency and reproducibility.